



Design Pattern Composite



Beatriz, Catarina e Kathleen.

Tópicos

1

O que é ?

2

Como funciona ?

3

Estrutura

4

Exemplo.

O que é

O Composite é um padrão de projeto estrutural que permite que você componha objetos em estruturas de árvores e então trabalhe com essas estruturas como se elas fossem objetos individuais.



Como funciona?

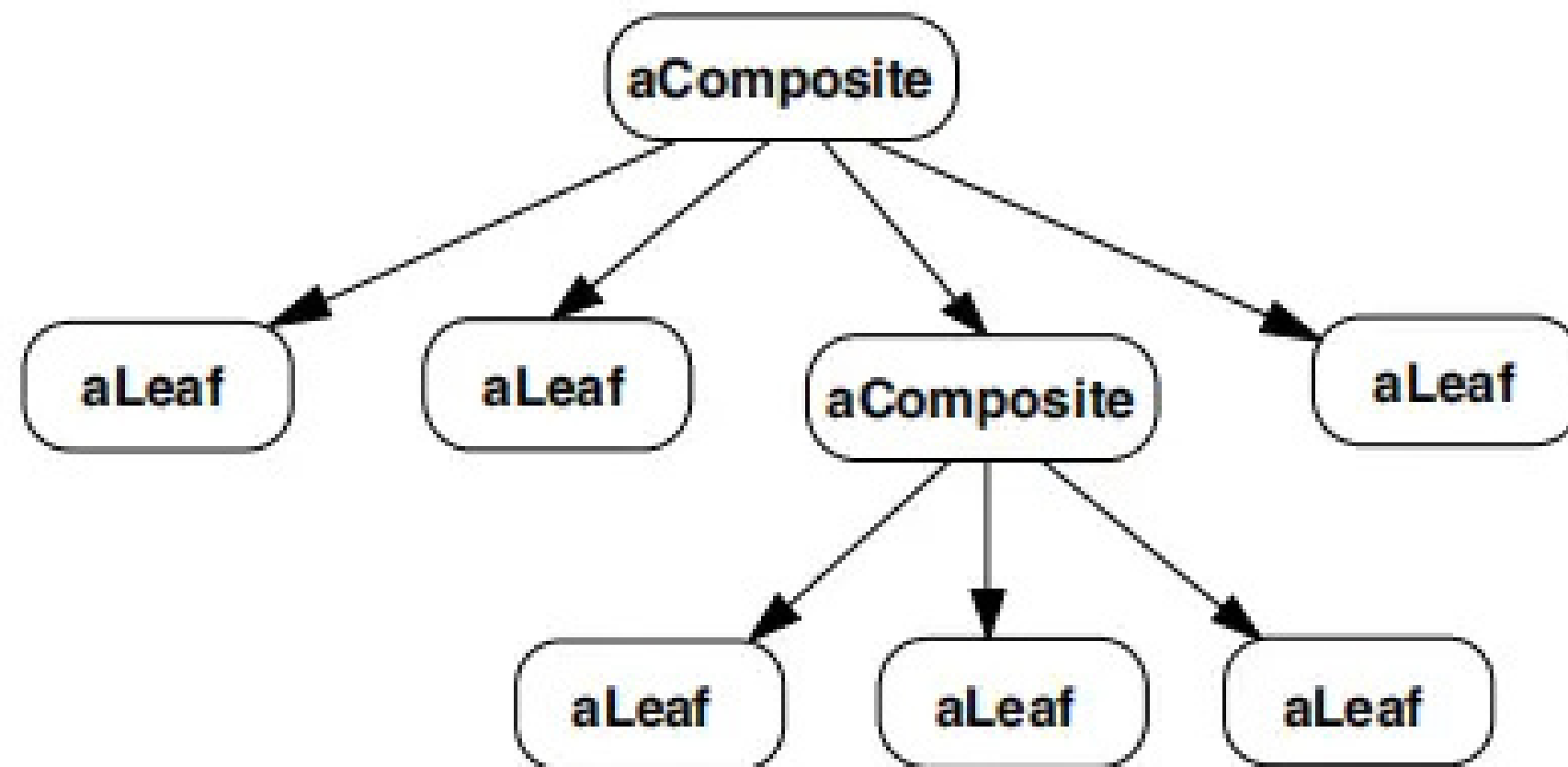
Dentro do Composite, temos uma classe abstrata conhecida como “Componente”, que estabelece uma interface compartilhada para objetos individuais e grupos de objetos.

Essa estrutura permite a realização de interações recursivas, e dessa forma possibilita a aplicação de operações a todos os objetos na hierarquia.

O padrão Composite permite tratar objetos individuais e grupos de forma uniforme, simplificando o código e aumentando a flexibilidade em estruturas hierárquicas.

Estrutura

Composite



- Component: Interface comum para todos os objetos, tanto folhas quanto composites.
- Leaf: Representa objetos individuais que não têm filhos.
- Composite: Representa grupos de objetos que podem conter outros objetos, permitindo operações recursivas.

Exemplo

O exemplo do supermercado ilustra o padrão Composite ao mostrar que um fardo de refrigerantes pode ser tratado como um único produto, cujo preço é a soma dos preços das unidades que contém, permitindo tratar itens individuais e composições de forma uniforme.

```
JS produto.js > ProdutoIndividual > constructor
1 // Componente
2 class Produto {
3   constructor(nome) {
4     this.nome = nome;
5   }
6
7   getNome() {
8     throw new Error("Método 'getNome' deve ser implementado");
9   }
10
11   getPreco() {
12     throw new Error("Método 'getPreco' deve ser implementado");
13   }
14 }
15
16 // Folha
17 class ProdutoIndividual extends Produto {
18   constructor(nome, preco) {
19     super(nome);
20     this.preco = preco;
21   }
22
23   getNome() {
24     return this.nome;
25   }
26
27   getPreco() {
28     return this.preco;
29   }
30 }
31
32 // Composição
33 class Fardo extends Produto {
34   constructor(nome) {
35     super(nome);
36     this.produtos = [];
37   }
38 }
```

```
JS produto.js > ProdutoIndividual
33 class Fardo extends Produto {
39   adicionarProduto(produto) {
40     this.produtos.push(produto);
41   }
42
43   removerProduto(produto) {
44     const index = this.produtos.indexOf(produto);
45     if (index > -1) {
46       this.produtos.splice(index, 1);
47     }
48   }
49
50   getNome() {
51     return this.nome;
52   }
53
54   getPreco() {
55     let precoTotal = 0;
56     for (const produto of this.produtos) {
57       precoTotal += produto.getPreco();
58     }
59     return precoTotal;
60   }
61 }
62
63 // Classe principal para testar
64 const main = () => {
65   // Criando produtos individuais
66   const lata1 = new ProdutoIndividual("Refrigerante Lata", 3.00);
67   const lata2 = new ProdutoIndividual("Refrigerante Lata", 3.00);
68   const lata3 = new ProdutoIndividual("Refrigerante Lata", 3.00);
69
70   // Criando um fardo de refrigerantes
71   const fardo = new Fardo("Fardo de Refrigerantes");
72   fardo.adicionarProduto(lata1);
73   fardo.adicionarProduto(lata2);
74 }
```

```
// Exibindo informações
console.log("Nome do fardo: " + fardo.getNome());
console.log("Preço total do fardo: R$ " + fardo.getPreco().toFixed(2));
};

// Executando a função principal
main();
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [versão 10.0.22631.5039]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\Instrutor\Desktop\composite>node produto.js
Nome do fardo: Fardo de Refrigerantes
Preço total do fardo: R\$ 9.00

C:\Users\Instrutor\Desktop\composite>

Prós e Contras

Prós

Simplifica a estrutura hierárquica

Oferece flexibilidade na manipulação de objetos e operações.

Promove reuso do código.

Contras

Pode ter impacto na eficiência em estruturas muito grandes

Pode exigir maior esforço de design e implementação inicial

Requer uma interface comum bem definida

Obrigada 🍷